

HYUNDAI MOTOR · 차량사이버보안개발팀

Day 1

Security by Design · Harness 기초

통합 운영자료 (개념+실습 병행) · 6/8 月 · 09:00–17:00

정책으로 통제되는 첫 보안 에이전트를 만든다 — 이론과 실습을 하루에 잇는다.

이 파일 하나로 하루 수업을 끝까지 진행합니다. 상세 이론은 *Appendix*를 참조하세요.

모두의연구소 · Agentic Security 심화 과정 · 2주차 (Day 1~5)

Day 1 운영 흐름

정책으로 통제되는 첫 보안 에이전트를 만든다 — 이론과 실습을 하루에 잇는다.

09:00-09:50	09:50 -	50m	개념+파일
10:50-12:00	12:00 -	70m	실행+확인
13:20-14:40	14:40 -		
16:10-17:00	16:10	50m	정리+산출물 확인

Day 1 — 무엇을 배우고 무엇을 만드는가

학습 목표·실습 목표·사용 파일·성공 기준을 시작 전에 못 박아 둡니다.

학습 목표 (개념·실습 병행)

- Agent와 RAG의 차이, 하네스 5요소
- CLAUDE.md 정책 주입의 원리
- 단일 Agent의 구조적 한계

실습 목표 (실습·산출물 확인)

- check_env.py로 환경 점검
- sast_agent.py 실행·정책 주입
- 정답지 9종 중 8종 탐지, 전체 9건 출력 확인

사용 파일

- check_env.py
- agents/sast_agent.py
- policies/CLAUDE_base.md
- policies/CLAUDE.md
- sample_app/ (vulnerability.py, control/)

성공 기준

- vulnerability.py 정답지 9종 중 8종 탐지 확인
- sample_app 전체 실행 시 control/ 포함 총 9건 출력
- CWE-532 민감 정보 로깅 누락 사유 설명
- control/ 발견의 High → Critical 정책 상향 확인

Day 1

5일 로드맵: 단일 Agent에서 통합 Security PoC까지

오늘이 전체 과정에서 어디에 위치하는지 먼저 확인합니다.

단일 Agent에서 통합 Security PoC까지 이어지는 하나의 파이프라인입니다.



① 개념 설명 → ② 파일 확인 → ③ 직접 실행 → ④ 산출물 확인 → ⑤ 관찰 정리 → ⑥ 다음 연결

개념 + 실습 병행 운영 패턴 (09:00-17:00)

개념을 설명하고 바로 파일 확인·실행·산출물 검증으로 이어갑니다.

수업은 이론을 몰아서 듣는 방식이 아니라, 작은 학습 루프를 하루에 3회 이상 반복합니다.



매일 최소 3회 이상 직접 실행



운영 메모: TDD 관점: 탐지 기대 결과를 먼저 정의하고, 정책 변경 후 결과가 바뀌는지 검증한다.

오늘의 자료 구조: PPT → 코드 → 명령 → 산출물

수강생이 지금 보는 장표가 어떤 파일과 어떤 결과물로 이어지는지 명확히 보여줍니다.

PPT의 개념은 실제 Codespaces 파일·명령어·산출물로 바로 연결되어야 합니다.



PPT 개념

정책으로 통제되는
단일 Agent

Harness · CLAUDE.md ·
SAST

확장 개념
Agent Loop, Policy Injection,
Output Schema



관련 파일

policies/CLAUDE_base.md

policies/CLAUDE.md

agents/sast_agent.py

sample_app/



실행 명령

```
python agents/sast_agent.py --target sample_app/  
--policy policies/CLAUDE_base.md
```

```
python agents/sast_agent.py --target sample_app/  
--policy policies/CLAUDE.md
```



산출물

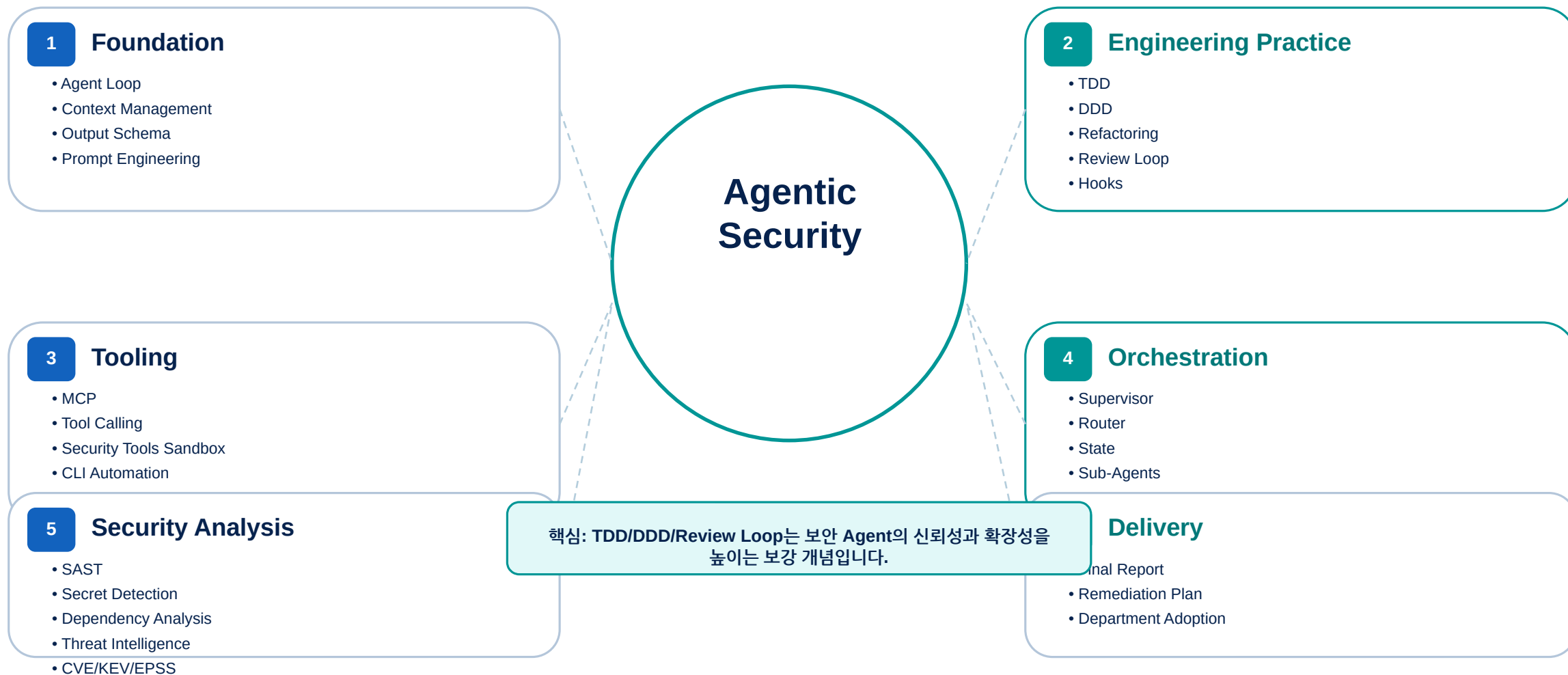
reports/lab1_sast.md

reports/lab1_schema.md

보강 개념 맵: 에이전틱 보안 엔지니어링

TDD·DDD·Review Loop·Hooks 등은 현재 실습 흐름을 보강하는 개념으로 연결합니다.

오늘 핵심 개념을 보안 실습 흐름에 맞춰 정리한 장입니다.



정책으로 통제되는 단일 Agent

이 그림은 오늘의 핵심 구조를 파일·실행·산출물 관점으로 연결합니다.



정책으로 통제되는 단일 Agent

Mini Lab → Main Lab 연결

작은 실험으로 개념을 먼저 체감하고, 바로 보안 실습으로 확장합니다.

Mini Lab · 회의 메모 정리 Agent

Single Agent
Policy File
Output Schema
meeting_summary.md

Main Lab · 정책 기반 SAST Agent

CLAUDE.md
SAST Agent
Secret Masking
lab1_sast.md

연결 문장

MEETING_AGENT.md가 출력 기준을 통제하듯, CLAUDE.md는 보안 Agent의 판단 기준을 통제합니다.

Mini Lab 실행 · 회의 메모 정리 Agent

한 줄 명령으로 실행하고, 산출물을 확인합니다.

왜 하는가

정책 파일이 Agent 결과를 어떻게 바꾸는지
보안 전 쉬운 예제로 확인합니다.

```
python course/mini_labs/day01_single_agent/meeting_agent.py \  
--input course/mini_labs/day01_single_agent/meeting_notes.txt \  
--policy course/mini_labs/day01_single_agent/MEETING_AGENT.md
```

관련 파일

course/mini_labs/day01_single_agent/
meeting_agent.py
meeting_notes.txt
MEETING_AGENT.md

생성 산출물

reports/meeting_summary.md

확인 포인트

Risks 섹션 생성 여부
정책 수정 후 출력 변화
CLAUDE.md와의 연결

TDD · Output Schema · Secret Masking 계약

부가 개념을 키워드가 아니라 실습·산출물·검증과 연결합니다.

Output Schema

자유 응답을 구조화된 산출물로 바꿉니다.
회의 메모는 Summary/Owners/Risks, SAST는 severity/CWE/evidence로 고정합니다.

TDD 관점

보안 정책은 말로만 두지 않습니다.
마스킹·스키마·기대 결과를 테스트로 잠가 회귀를 막습니다.

Secret Masking

리포트는 공유되는 산출물입니다.
비밀값 원문은 reports/와 memory/에 남지 않아야 합니다.

Agent vs RAG — 무엇이 다른가

1주차의 RAG와 2주차의 주인공 에이전트의 결정적 차이를 한 장으로 짚습니다.

RAG (1주차)

- 질문 → 검색 → 답변의 1회성 흐름
- 스스로 도구를 부르거나 행동하지 않음
- 상태를 들고 다니지 않음

Agent (2주차)

- 인식→추론→행동→관찰을 스스로 순환
- 도구를 직접 호출하고 환경을 바꿈
- 상태를 누적하며 다단계로 일함

RAG는 사라지지 않습니다. 정책·위협 정보를 '참조'하는 보조 소스로 에이전트 안에 들어옵니다. 상세는 Appendix A-1.

차량보안에서 Agent가 필요한 이유

추상적 정의가 아니라, 우리 팀 업무에서 에이전트가 어디에 쓰이는지로 연결합니다.

①

반복적 코드 점검

PR마다 반복되는 SAST·시크릿·의존성 점검을 일관된 기준으로 자동 수행합니다.

②

방대한 공격 표면

ECU·게이트웨이·앱 등 넓은 표면의 1차 스크리닝을 에이전트가 맡습니다.

③

위협 정보 결합

발견을 CVE·KEV·EPSS와 자동 대조해 '무엇부터 고칠지'를 제안합니다(Day 4).

④

감사 추적

누가·언제·무엇을·왜 점검했는지 자동 기록해 규제 대응(R155 맥락)의 근거를 남깁니다.

하네스 엔지니어링 5요소

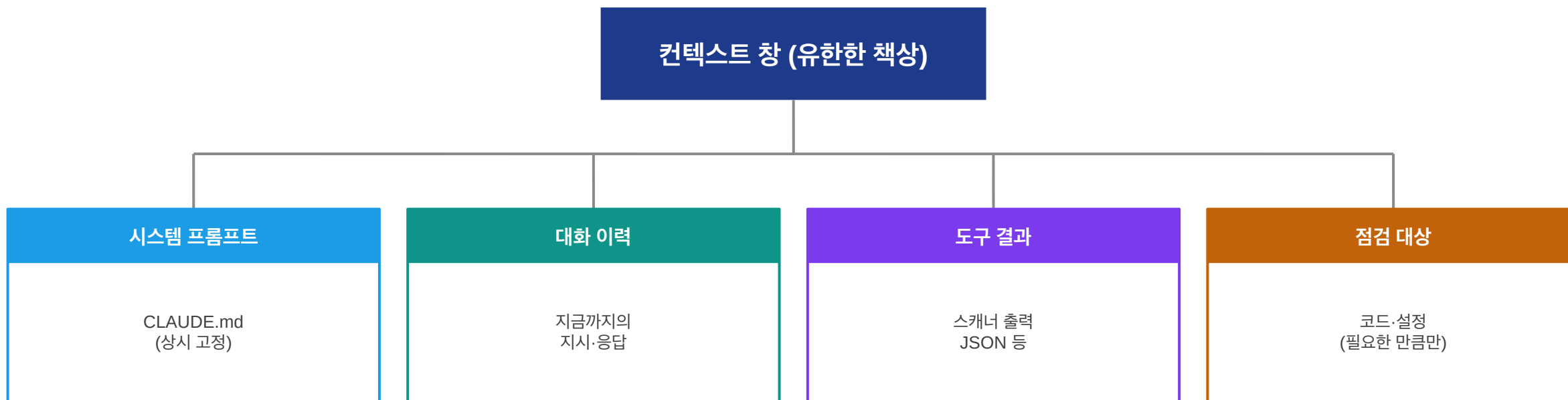
에이전트를 '쓰는 법'이 아니라 '통제하며 쓰는 법'. 이 과정의 중심 뼈대입니다. 요소별 상세는 Appendix A-2.



오늘은 ①시스템 프롬프트(CLAUDE.md)를 직접 만듭니다. ②~⑤는 Day 2~5에서 코드로 다룹니다.

그림으로 보는 컨텍스트 창 — 에이전트의 '책상'

에이전트가 한 번에 볼 수 있는 작업 공간은 유한합니다. 그 책상 위에 무엇이 올라가는지 그림으로 봅니다.



책상이 좁을수록 무엇을 올리고 내릴지가 중요해집니다. 관련 없는 것을 올리면 정작 필요한 것이 밀려납니다 — 이것이 컨텍스트 관리의 본질입니다.

CLAUDE.md 잘 쓰는 법 — 네 칸 구조

정책 파일을 처음 쓸 때 막막하지 않도록, 실무에서 통하는 네 칸 구조를 소개합니다.

①

역할 선언

'너는 차량 SW 보안 점검 에이전트다'처럼 정체성을 한 문장으로 못 박습니다. 모든 판단의 기준점이 됩니다.

②

해야 할 것

긍정문 규칙을 적습니다. '모든 발견에 CWE와 근거를 붙인다'처럼 행동을 지시하는 문장이 금지문보다 잘 작동합니다.

③

하지 말 것

꼭 필요한 금지문 짧게. '비밀 값 원문을 출력하지 않는다'처럼 위반 시 피해가 큰 것 위주로 적습니다.

④

출력 형식

결과물의 모양을 정합니다. 형식이 고정되면 후속 도구(리포트 생성 등)가 안정적으로 이어받습니다.

요령: 짧게 시작해 결과를 보며 한 줄씩 다듬습니다. 정책은 한 번에 완성되는 문서가 아니라, 점검 결과와 함께 자라는 문서입니다.

CLAUDE.md 정책 주입의 원리

하네스 요소①의 실천입니다. 정책이 에이전트의 판단에 어떻게 작용하는지 원리를 봅니다.

원리

상시 적용되는 최상위 규칙

CLAUDE.md의 문장은 에이전트가 매 판단마다 참조하는 헌법입니다. 사내 규정을 '문장'으로 옮기면 곧 에이전트의 행동 기준이 됩니다.

예

심각도 가중

'차량 제어 모듈의 발견은 한 단계 높은 심각도로 보고한다'는 한 줄이, 같은 발견의 심각도를 High에서 Critical로 바꿉니다.

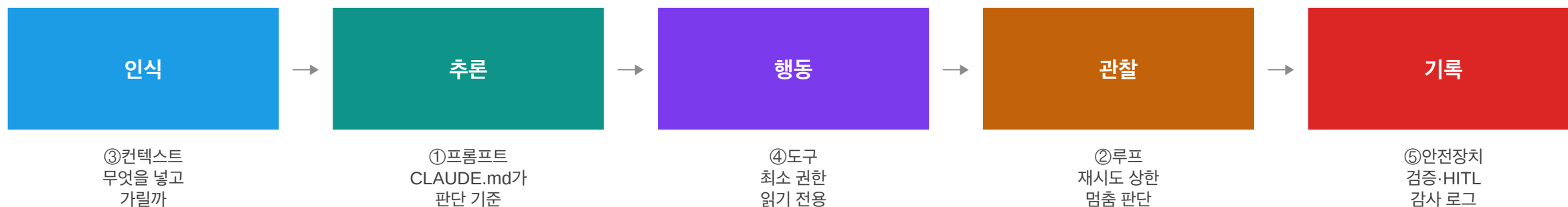
확인

실습 중 직접

실습 LAB 1에서 이 문장을 넣고 빼며 sample_app/control/ 발견의 심각도 변화를 눈으로 확인합니다.

그림으로 보는 통제 지점 — 루프 위의 하네스

에이전트 루프의 각 단계에 하네스 요소가 어디서 개입하는지 한 장으로 묶습니다.



하네스 5요소는 따로 노는 목록이 아니라, 루프의 각 단계를 하나씩 맡아 감싸는 '배치도'입니다. 이 그림 한 장이 본 과정 전체의 지도입니다.

단일 Agent의 한계

오늘 LAB 1에서 직접 관찰할 한계를 미리 개념으로 정리합니다. 이것이 Day 3 멀티의 동기입니다.

①

범위가 넓어질 때

코드+의존성+설정을 한 에이전트에 다 맡기면 분석 깊이가 얕아집니다.

②

역할이 섞일 때

SAST·시크릿·위협 조회를 한 번에 시키면 각 영역의 근거 품질이 떨어집니다.

③

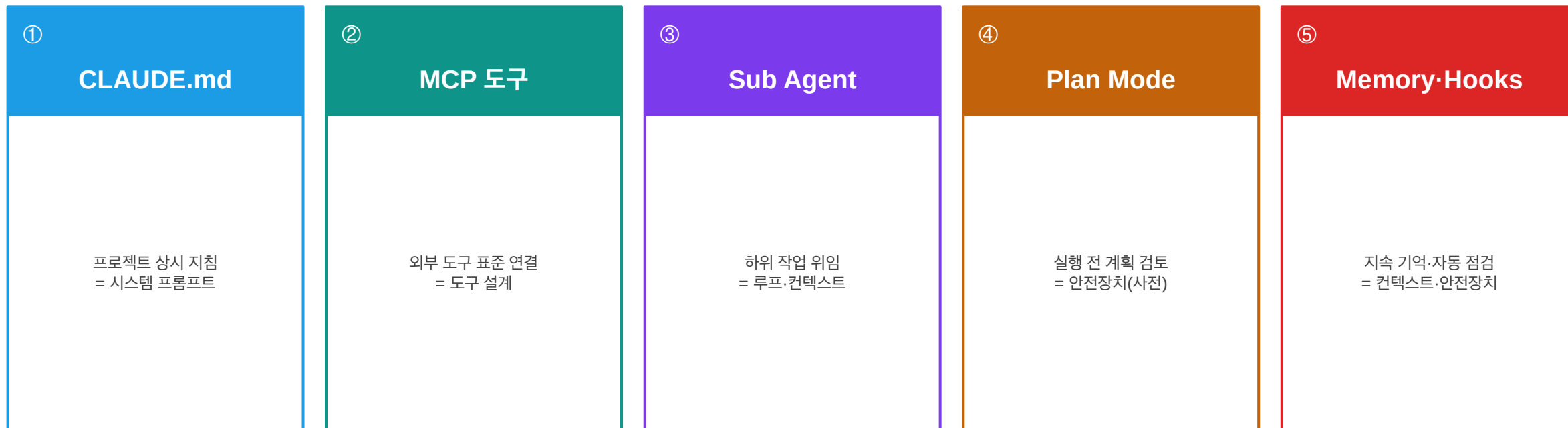
검증 주체가 없을 때

자기 결과를 자기가 검증하면 오류를 놓칩니다. 별도 검증 역할이 필요합니다.

핵심: 이것은 '성능 문제'가 아니라 '구조 문제'입니다. 더 좋은 모델이 아니라 분업이 답입니다. 상세는 Appendix A-3.

Claude Code의 통제 기능 — 하네스로 읽기

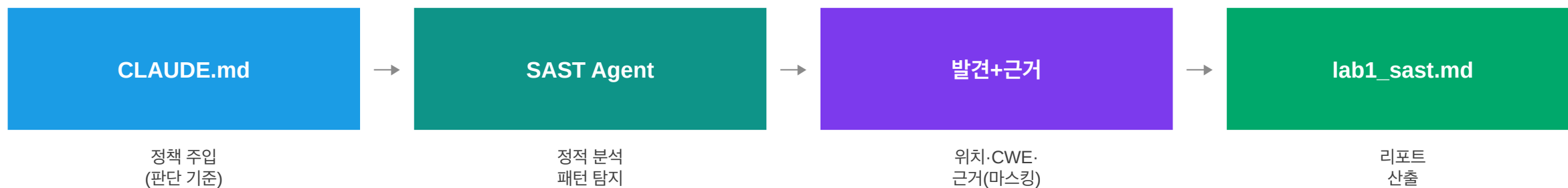
에이전틱 코딩 도구가 제공하는 통제 장치들은 하네스 5요소와 그대로 짝지어집니다. 공개 문서 기준으로 정리합니다.



본 과정은 특정 도구 사용법 암기가 목적이 아닙니다. 어떤 에이전틱 코딩 도구를 쓰든, 이 기능들을 '하네스 5요소'라는 통제 틀로 읽어내는 눈을 기릅니다.

그림으로 보는 Day 1 흐름 — 정책에서 리포트까지

오늘 만든 것을 하나의 흐름으로 잇습니다. 정책 한 줄이 어떻게 최종 리포트까지 영향을 미치는지 봅니다.



정책(CLAUDE.md)을 바꾸면 가장 왼쪽 한 칸의 변화가 오른쪽 끝 리포트의 심각도까지 바꿉니다. 이것이 '정책의 코드화'의 파급력입니다.

산출물 미리보기 — lab1_sast.md는 이렇게 생겼다

오늘 만드는 리포트의 실제 구조입니다. 발견마다 위치·CWE·근거가 따라붙고, 비밀값은 마스킹됩니다.

reports/lab1_sast.md (구조)

```
# LAB 1 - SAST 단일 분석 리포트
총 발견: 9건
- vulnerability.py 기준: 정답지 9종 중 8종 탐지
- control/ 기준: 차량 제어 모듈 발견 1건 추가
- 미탐지: CWE-532 민감 정보 로깅

- **[Critical]** CWE-78 `control/ecu_handler.py:13`
  - 근거: `subprocess.call(cmd, shell=True)`
- **[High]** CWE-798 `vulnerability.py:31`
  - 근거: `API_TOKEN = "sk-****"` ← 마스킹됨
```

근거 코드의 비밀값이 sk-**** 로 가려진 것에 주목하세요. 정책이 가르친 원칙을 도구가 실제로 지킵니다.

실습 루프 (LAB 0)

LAB 0 — GitHub Codespaces 환경 점검

여기서부터 실습·산출물 확인입니다. 전원이 같은 출발선에 서는 것이 목표입니다.

LAB 0가 하는 일 — 한눈에

환경 점검의 목표와 준비물, 사용 파일, 성공 신호를 먼저 정리합니다.

목표

- 전원이 동일한 클라우드 환경 확보
- check_env.py 항목 통과

준비물

- GitHub 계정 + Repo 권한
- Claude Code/Codex 로그인
- API 키 (Secrets 권장)

사용 파일

- check_env.py
- .env.example
- .devcontainer/

성공 신호

- '환경 점검 통과' 출력
- LAB 1로 진행 가능

Codespace 생성 — 화면 단계

처음 만드는 분도 헤매지 않도록 버튼 위치와 대기 시간을 짚습니다.

1

Code 버튼

저장소 우측 상단 초록색 'Code' 버튼 → 'Codespaces' 탭으로 이동합니다.

2

Create codespace

'Create codespace on main'을 누르면 클라우드에 개인 개발 환경이 생성됩니다.

3

자동 구성 대기

약 2분간 Python 3.12 설치와 의존성 구성(devcontainer)이 자동 진행됩니다.

4

VS Code 화면

브라우저 안에 VS Code가 열립니다. 로컬 설치 없이 모두 동일 화면에서 실행합니다.

별도 서버나 포트 실행이 없습니다. 버튼 한 번이면 환경이 완성됩니다.

API 키 설정 — 두 가지 안전한 방법

키는 절대 코드에 적지 않습니다. 키를 코드에 두지 않는 것 자체가 첫 번째 보안 실습입니다.

방법 A · Codespaces Secrets (권장)

```
# GitHub → Settings → Codespaces → Secrets 에 등록
# ANTHROPIC_API_KEY = sk-ant-... (또는 OPENAI_API_KEY)
# 등록 후 Codespace 재시작하면 환경변수로 주입됨
```

방법 B · .env 파일

```
cp .env.example .env # 템플릿 복사
# .env 에 키 입력 (따옴표 없이 한 줄)
# .gitignore가 .env를 자동 제외 → 커밋 사고 방지
```

방법 A가 더 안전합니다. 키가 파일로 남지 않기 때문입니다.

환경 점검 실행 — python check_env.py

ZIP 루트의 check_env.py를 그대로 실행합니다. 다섯 항목을 한 번에 점검합니다.

실행

```
python check_env.py
```

정상 통과 출력 (Codespaces 기준)

```
[OK] Codespaces runtime
[OK] repo root
[OK] claude CLI
[WARN] codex CLI 미확인 — Codex 선택 실습이면 설치 필요 (미사용 시 무시)
[OK] Python 3.12 / [OK] packages
[WARN] API key 미설정 — 도구 분석은 가능, LLM 해석은 건너뛸
[OK] mcp_servers/security_tools_server.py exists / [OK] sample_app
환경 점검 통과 — LAB 1로 진행하세요.
```

주의: 이 항목은 파일 존재 확인입니다. 실제 등록(claude mcp add security-tools -- python mcp_servers/security_tools_server.py)은 Day 2에서 수행합니다.

환경 점검이 실패하면 — 항목별 대응

가장 흔한 두 실패와 즉시 대응법입니다.

[FAIL] packages

- 원인: 의존성 설치 완료 전 실행
- 조치: `pip install -r requirements.txt`
- 1~2분 후 재실행

[FAIL] mcp_servers/security_tools_server.py

- 원인: 파일 경로 문제/얕은 클론
- 조치: `ls mcp_servers/security_tools_server.py`
- 저장소 다시 클론하면 해결

API key [WARN]은 실패가 아닙니다. 도구 기반 분석은 키 없이 동작하므로 그대로 진행합니다.

실습 루프 (LAB 1)

LAB 1 — 단일 SAST Agent

환경이 갖춰졌으니, 정책에 통제되는 첫 번째 보안 분석가를 직접 돌려봅니다.

오늘의 실습 계단 — LAB 1-1 → 1-2 → 1-3

단일 SAST Agent를 세 단계로 다룹니다. 한 계단씩 오르면 '정책으로 통제되는 에이전트'가 손에 잡힙니다.

1-1

단일 SAST 실행

base/full 정책으로
두 번 실행해
전후 비교

1-2

정책 튜닝

CLAUDE.md에
내 정책 한 줄
추가·수정

1-3

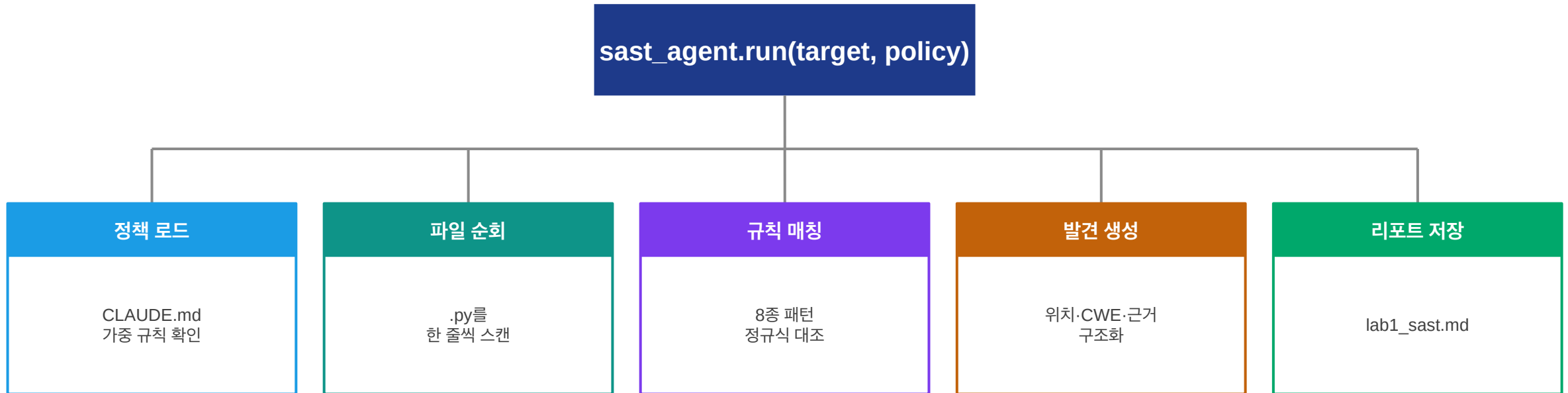
출력 스키마 고정

발견을 고정 형식
(finding_id~
recommendation)으로

이 계단이 내일로 이어집니다: 1-3에서 고정한 출력 형식이 Day 2 도구 표준화의 토대가 되고, Day 3에서 여러 Agent의 결과를 한 State에 모을 수 있게 합니다.

SAST Agent 개요 — 안에서 무슨 일이 일어나나

실행 명령 한 줄 뒤의 동작을 그림으로 봅니다.



별도 서버나 포트 실행 없이, 단일 파이썬 함수로 분석이 수행됩니다. Day 3에서 이 함수가 '그래프 노드'가 됩니다.

SAST Agent 실행 — ZIP의 실제 명령 그대로

정책 파일을 인자로 넘겨 실행합니다. 이 명령은 ZIP 안의 실제 파일·인자와 1:1로 일치합니다.

실행

```
python agents/sast_agent.py --target sample_app/ --policy policies/CLAUDE.md
```

출력 (요약)

```
[SAST] sample_app/ 분석 완료 - 발견 9건 (정책: policies/CLAUDE.md)
[Critical] CWE-78   control/ecu_handler.py:13  Command Injection 의심
[High     ] CWE-798 vulnerability.py:31  하드코딩 자격증명
[High     ] CWE-89   vulnerability.py:39  SQL Injection 의심
[High     ] CWE-502 vulnerability.py:74  안전하지 않은 역직렬화
[Medium   ] CWE-328 vulnerability.py:64  약한 해시 사용
→ sample_app 전체: vulnerability.py 8건 + control/ 1건 = 총 9건
```

기준: vulnerability.py 정답지 9종 중 8종 탐지(8건), sample_app 전체 실행 시 control/ 1건 포함 총 9건 출력. CWE-532 1종은 미탐지(Day 3 개선).

결과 읽기 — 확인할 네 가지

출력을 그냥 지나치지 말고 네 가지를 체크리스트 삼아 읽습니다. 이 습관이 Day 3 검증 루프의 밑천입니다.

①

발견의 구조

위치·CWE·심각도·근거가 모두 있는가? 근거 없는 발견은 검증할 수 없는 발견입니다.

②

확정과 의심

'확정'과 '의심'을 구분해 표기하는가? 구분이 없으면 위양성 관리가 어렵습니다.

③

탐지율

정답지 9종 중 몇 건이 잡혔는가? 8건 탐지, 1건(CWE-532) 누락을 직접 확인합니다.

④

재현성

같은 명령을 두 번 돌리면 결과가 같은가? 변동이 있다면 어디서 오는가?

정책 주입 — 두 정책 파일로 전/후를 나눈다

재현 가능한 비교를 위해, 가중 규칙이 없는 CLAUDE_base.md와 있는 CLAUDE.md를 따로 둡니다.

policies/CLAUDE_base.md (전: 가중 규칙 없음)

점검 원칙 (기본)

- 모든 발견에 CWE·근거 포함, 비밀 값 마스킹 ... (가중 규칙 없음)

policies/CLAUDE.md (후: 가중 규칙 포함)

심각도 가중 규칙

- 차량 제어 관련 모듈(control/)의 발견은 한 단계 높은 심각도로 보고한다.

두 파일을 번갈아 --policy로 넘기면, 같은 발견의 심각도가 어떻게 달라지는지 재현 가능하게 비교됩니다.

정책 전후 비교 — 같은 명령, 다른 정책 파일

정책 파일만 바꿔 두 번 실행하고, control/ 발견의 심각도 변화를 확인합니다. ZIP 명령과 1:1 일치.

① 정책 적용 전 (CLAUDE_base.md)

```
python agents/sast_agent.py --target sample_app/ --policy policies/CLAUDE_base.md  
[High    ] CWE-78  control/ecu_handler.py:13  Command Injection 의심
```

② 정책 적용 후 (CLAUDE.md)

```
python agents/sast_agent.py --target sample_app/ --policy policies/CLAUDE.md  
[Critical] CWE-78  control/ecu_handler.py:13  Command Injection 의심  
← 차량 제어 모듈이므로 High → Critical 자동 상향
```

CLAUDE.md의 '차량 제어 관련 모듈은 한 단계 높은 심각도로 보고한다' 정책이 High를 Critical로 상향합니다.

LAB 1-2 — CLAUDE.md 정책 튜닝

수강생이 직접 정책 문장을 추가·수정하고, 결과 변화를 자동 비교 스크립트로 확인합니다.

목표

- 사내 규정을 정책 문장으로 옮기기
- control/ 심각도 상향 확인
- 비밀값 원문 출력 금지 확인

실행 명령

- `python scripts/run_lab1_policy_compare.py --target sample_app/`
- (base와 full을 한 번에 비교)

생성 산출물

- `reports/lab1_policy_compare.md`
- 수정한 `policies/CLAUDE.md`

관찰 포인트 · 연결

- `ecu_handler.py` High → Critical 상향 1건
- 리포트 근거의 `sk-****` 마스킹
- → Day 3에서 이 정책이 5개 Agent 전체에 적용됨

LAB 1-3 — 출력 형식(스키마) 고정

발견을 고정 스키마로 출력합니다. 형식이 고정되어야 다음 단계가 코드로 이어받습니다. ZIP 명령과 1:1 일치.

실행

```
python scripts/run_lab1_schema_demo.py --target sample_app/
```

출력 · 스키마 (reports/lab1_schema.md)

```
## SAST-001
- severity: High          - cwe: CWE-798
- file: vulnerability.py - line: 31
- evidence: `API_TOKEN = "sk-****" # noqa`
- recommendation: 비밀을 코드에서 분리해 .env/Secrets로 이동, 키 회전
```

관찰 포인트: 모든 발견이 같은 칸(finding_id·severity·cwe·file·line·evidence·recommendation)에 담깁니다. 이 고정 형식이 Day 2 도구 출력(JSON)과 Day 4 점수화의 토대입니다.

정답지 대조 — 무엇을 놓쳤는가

몇 건을 잡았는지보다 '무엇을 놓쳤는지'를 아는 것이 점검의 핵심입니다.

①

정답지 확인

vulnerability.py에 심어진 9종이 채점 기준입니다. (SQLi·하드코딩·CMDi·경로·해시·역직렬화·디버그·리다이렉트·민감로깅)

②

탐지 건수

기본 SAST Agent는 이 중 8종을 탐지합니다. 출력에서 직접 세어 확인합니다.

③

누락 식별

누락 1건은 CWE-532(민감 정보 로깅)입니다. 단순 패턴으로는 '정상 로깅'과 구분이 어렵기 때문입니다.

④

개선 예고

이 누락 1건을 Day 3에서 규칙·LLM 보강으로 개선합니다. 오늘은 '무엇을 왜 놓쳤는가'를 기록만 합니다.

핵심 학습 포인트: 정답지 9종 삽입 → 8건 탐지 → 1건 누락 식별 → Day 3 개선. 이 흐름 자체가 오늘의 결론입니다.

단일 한계 관찰 — 직접 부딪혀 보기

일부러 무리한 요구를 던져 단일 구조의 한계를 관찰합니다. 이 경험이 Day 3 '왜 멀티인가'의 답입니다.

관찰1 범위 넓히기
--target을 더 넓혀 한 에이전트에 더 많은 것을 맡기면 분석 깊이가 어떻게 알아지는지 봅니다.

관찰2 누락의 의미
CWE-532를 놓친 것은 '성능' 문제가 아니라, 단일 패턴 규칙의 '구조적' 한계입니다.

관찰3 측정값 메모
탐지율·분석 시간을 적어 둡니다. Day 3에서 멀티 분업으로 다시 재고 비교합니다.

Day 1 — 오늘 만든 산출물 확인

오늘 저장소에 실제로 남은 것들입니다. 다음 Day의 입력으로 누적됩니다.

01

reports/lab1_sast.md

단일 SAST Agent의 발견 리포트 — vulnerability.py 기준 정답지 9종 중 8종 탐지 · sample_app 전체 기준 control/ 1건 포함 총 9건 · 근거 코드의 비밀값은 마스킹되어야 함

02

수정한 policies/CLAUDE.md

차량 제어 모듈 가중 정책이 주입된 시스템 프롬프트

두 산출물은 이후 Day에도 계속 쓰입니다. CLAUDE.md는 Day 3에서 각 Agent에 그대로 전달됩니다.

Day 1 — 자주 나는 오류

강의장에서 가장 흔히 막히는 지점과 즉시 대응법입니다.

ModuleNotFoundError: agents

→ 저장소 루트에서 실행하거나 PYTHONPATH를 루트로 지정

Agent와 RAG를 모두 '답변 생성'으로만 이해함

→ 에이전트의 핵심은 순환·행동·도구 호출임을 다시 짚기

하네스를 '프롬프트 작성'으로만 좁게 봄

→ 하네스는 프롬프트·루프·컨텍스트·도구·안전장치 5요소 전체

단일 Agent 한계를 성능 문제로 오해

→ 더 좋은 모델이 아니라 분업(구조)이 답 — 구조 문제로 이해

정책 바뀌도 결과 동일

→ --policy 경로·파일 저장 확인, control/ 경로 발견인지 확인

Day 1 — 완료 체크리스트 · 다음 Day 연결

체크리스트로 마감하고, 내일로 이어지는 연결고리를 확인합니다.

✓ check_env.py 전 항목 통과 (claude --version 포함, Codex 사용 시 codex --version)

✓ vulnerability.py 9종 중 8종 탐지 확인

✓ sample_app 전체 총 9건 출력 확인

✓ CWE-532 누락 사유 설명

✓ CLAUDE_base.md → CLAUDE.md 정책 전후 비교

✓ control/ High → Critical 상향 확인

✓ 리포트 근거 코드에서 비밀값이 마스킹되어 있는지 확인

다음 Day 연결고리 → Day 2: LLM의 눈으로만 보던 SAST에 MCP로 결정적 보안 도구를 쥐어준다

APPENDIX

부록 — 이론 상세 보충

본문에서 압축한 개념의 상세 설명입니다. 수업 중 질문이 나오거나, 복습 시 참조하세요.

[상세] 챗봇 · RAG · 에이전트 전체 비교

본문 '이론 1'의 상세판입니다. 세 가지를 같은 기준으로 펼칩니다.

기준	챗봇	RAG	에이전트
행동 능력	없음(답변만)	없음(검색+답변)	있음(도구 실행)
흐름	1턴	1턴(+검색)	다단계 순환
상태	없음	없음	누적
도구	없음	검색기 1종	여러 도구 자율 선택
보안 점검 적합도	낮음	보조	핵심 실행 주체

보안 점검은 '찾고 → 검증 → 우선순위 → 기록'의 다단계 작업이므로 에이전트가 적합합니다.

Appendix A-2

[상세] 하네스 5요소 — 요소별 깊이 보기

본문 '이론 3'의 상세판입니다. 다섯 요소를 정의·차량보안 적용·흔한 오해로 펼칩니다.

[상세] 요소 ① 시스템 프롬프트

에이전트의 모든 판단에 상시 적용되는 최상위 규칙. 본 과정에서는 policies/CLAUDE.md입니다.

정의 무엇인가
매 판단마다 참조하는 역할·원칙·금지사항의 묶음. 사람의 '직무 기술서 + 행동 강령'에 해당합니다.

**차량
보안** 적용 예
'차량 제어 모듈은 한 단계 높은 심각도', '비밀 값은 원문 출력 금지' 같은 사내 규정을 문장으로 주입합니다.

오해 흔한 오해
'한 번 잘 쓰면 끝'이 아닙니다. 정책은 새 위협·새 규정에 맞춰 계속 고쳐지는 살아있는 문서입니다.

[상세] 요소 ② 루프 · ③ 컨텍스트 관리

순환의 통제와 기억의 관리. 보안에서는 '무엇을 넣지 않을지'가 특히 중요합니다.

②

루프

순환의 통제

인식→추론→행동→관찰의 반복. 무한 루프를 막는 재시도 상한, 위험 행동 앞의 HITL이 통제점입니다.

③

컨텍스트

기억과 망각

작업 기억에 무엇을 넣고 뺄지. 사내 비밀, 고객 데이터는 마스킹해 외부 LLM으로 넘기지 않습니다.

오해

흔한 오해

'많이 넣을수록 똑똑'은 틀립니다. 관련 없는 컨텍스트는 판단을 흐립니다(컨텍스트 오염).

[상세] 요소 ④ 도구 설계 · ⑤ 안전장치

에이전트의 '손'과 '브레이크'. 권한을 좁히고, 멈춤과 기록을 심습니다.

④

도구

권한과 인터페이스

읽기 전용 점검 도구만 부여하고, 코드 수정·배포 권한은 주지 않습니다(최소 권한 원칙).

⑤

검증

검증 루프

결과가 기준을 만족하는지 자동 판정(PASS/RETRY/BLOCK). 근거 없는 발견은 다시 분석시킵니다.

⑤

감사

HITL·감사

위험 결정은 사람 승인(기본값 거부), 모든 결정은 시계열 기록(append-only). Day 3에서 코드로 구현합니다.

[상세] 일을 나누는 세 가지 배치 방식

본문 '이론 5'의 상세판입니다. 단일의 한계를 푸는 세 구조를 비교합니다.

① 어셈블리 라인

단계별 직선 흐름.
예측 가능·단순.

Prompt Chaining

② 허브앤스포크

중앙이 위임·취합.
본 과정의 Supervisor.

Orchestrator-Workers

③ 팀 협업

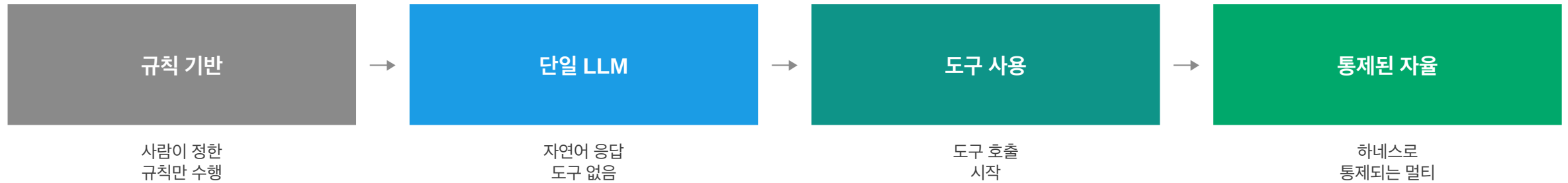
자율 협력.
자유롭지만 조율 비용 ↑.

(본 과정 미채택)

본 과정은 ② 허브앤스포크를 채택합니다. 모든 호출이 중앙을 지나야 검증·HITL·감사를 한곳에 둘 수 있기 때문입니다(Day 3).

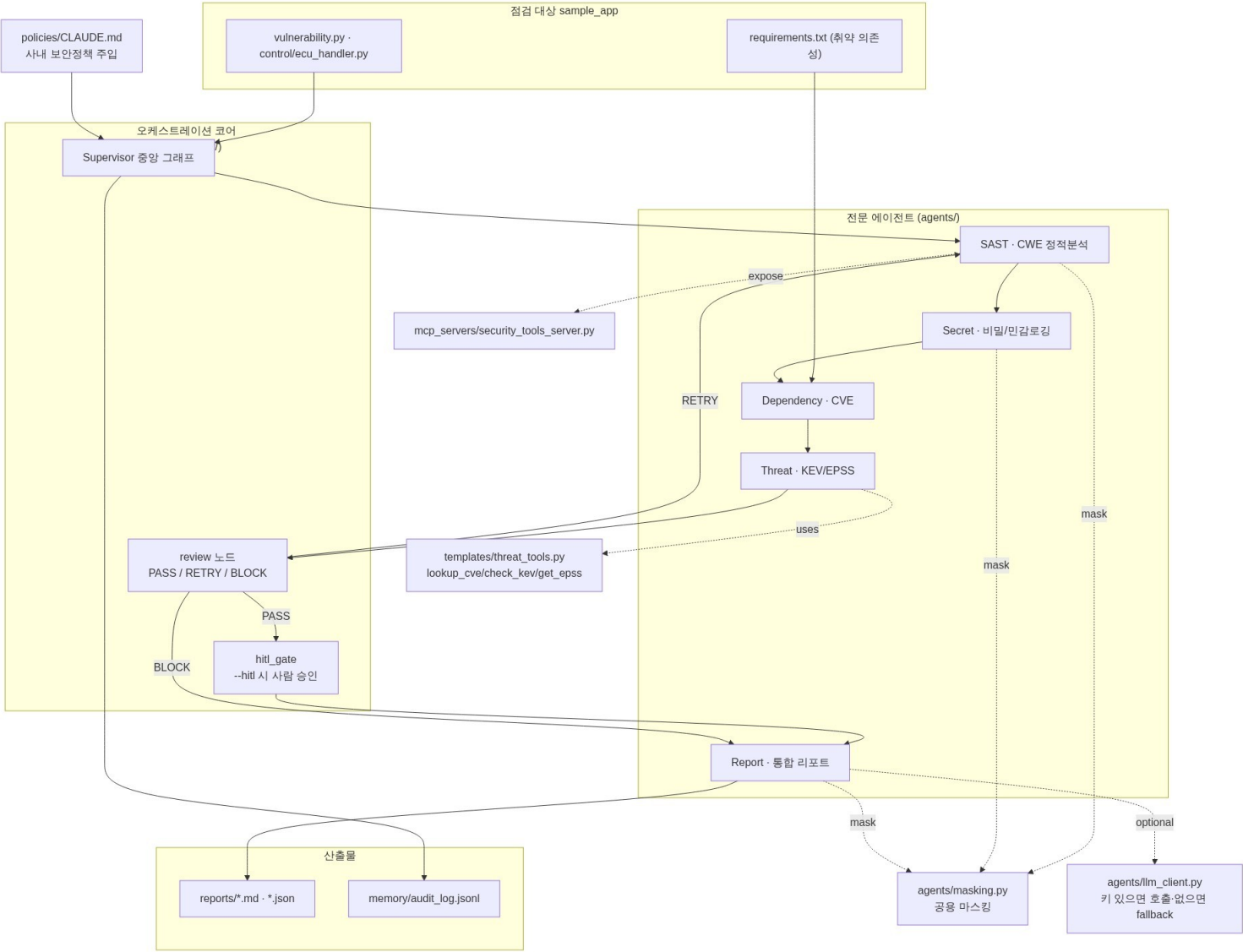
[상세] 에이전트 4단계 진화 — 우리의 좌표

규칙 기반에서 통제된 자율까지. 본 과정의 목표 지점을 확인합니다.



목표 지점은 4단계 '통제된 자율'입니다. 자율성 자체가 아니라 '통제'가 핵심어입니다.

전체 아키텍처 — Security Agent Lab



Day 1 동작 흐름 (시퀀스 다이어그램)

